实验6：Socket通信编程实验

实验内容

1. **TCP套接字编程**：实现客户端与服务器之间的基本通信功能（即一个客户端与一个服务器之间能进行简单的文字通信）。**【占本次实验总分的50%】**
2. **基于TCP的聊天程序设计**：参照实验课教材书的102-103页的要求，实现一个具有一定功能的聊天程序。所实现的聊天程序可涉及多种功能，比如：文字消息发送、文件传输、多人聊天、客户登录等。**【占本次实验总分的50%】**
3. 在聊天程序的实现过程中，可**发挥自己的想象力**自由选择实现方式：可以通过终端来演示聊天程序的交互过程，**但鼓励通过可视化方式演示【可在本次实验总分上再额外加分】**。

语言选择：Python、C++、Java都可以，可视化界面可以通过Qt、C#实现，自由选择。【关于TCP套接字编程的具体实现，可参考以下实验教程。如果是打算用python实现，还可以参考理论课教材书的第2.7节及相关课件内容；如果是打算用C/C++实现，还可以参考实验课教材书的第3章及其课件。】

分组方式：本次实验以小队为单位。一般情况下是**四人一队**。对于以队伍形式完成的实验，需要在实验报告里写清队内成员分工情况。

提交方式：将实验报告、源代码、可执行程序、演示视频（如果觉得实验报告里的文字和截图不太好呈现聊天程序的动态效果，可以录制一个简单的演示视频）等材料打包成一个压缩包。每支队伍由队长负责提交。

实验目的

- 掌握TCP和UDP协议主要特点和工作原理
- 理解socket的基本概念和工作原理
- 编程实现socket网络通信（C/Python/Java）

实验原理

一、什么是Socket

socket即套接字，用于描述地址和端口，是一个通信链的句柄。应用程序通过socket向网络发出

请求或者回应。

sockets(套接字)编程有三种，流式套接字（SOCK_STREAM），数据报套接字（SOCK_DGRAM），原始套接字（SOCK_RAW）；前两种较常用。基于TCP的socket编程是采用的流式套接字。

（1）SOCK_STREAM表示面向连接的数据传输方式。数据可以准确无误地到达另一台计算机，如果损坏或丢失，可以重新发送，但效率相对较慢。常用的HTTP协议就使用SOCK_STREAM传输数据，因为要确保数据的正确性，否则网页不能正常解析。

（2）SOCK_DGRAM表示无连接的数据传输方式。计算机只管传输数据，不作数据校验，如果数据在传输中损坏，或者没有到达另一台计算机，是没有办法补救的。也就是说，数据错了就错了，无法重传。因为SOCK_DGRAM所做的校验工作少，所以效率比SOCK_STREAM高。

QQ视频聊天和语音聊天就使用SOCK_DGRAM传输数据，因为首先要保证通信的效率，尽量减小延迟，而数据的正确性是次要的，即使丢失很小的一部分数据，视频和音频也可以正常解析，最多出现噪点或杂音，不会对通信质量有实质的影响。

注意：SOCK_DGRAM没有想象中的糟糕，不会频繁的丢失数据，数据错读只是小概率事件。

有可能多种协议使用同一种数据传输方式，所以在socket编程中，需要同时指明数据传输方式和协议。

二、面向连接的客户端/服务端模式：

在TCP/IP网络应用中，通信的两个进程相互作用的主要模式是客户/服务器模式，即客户端向服务器发出请求，服务器接收请求后，提供相应的服务。客户/服务器模式的建立基于以下两点：

（1）建立网络的起因是网络中软硬件资源、运算能力和信息不均等，需要共享，从而就让拥有众多资源的主机提供服务，资源较少的客户请求服务这一非对等作用。

（2）网间进程通信完全是异步的，相互通信的进程间既不存在父子关系，又不共享内存缓冲区。

因此需要一种机制为希望通信的进程间建立联系，为二者的数据交换提供同步，这就是基于客户/服务端模式的TCP/IP。

服务端：建立socket，声明自身的端口号和地址并绑定到socket，使用listen打开监听，然后不断用accept去查看是否有连接，如果有，捕获socket，并通过recv获取消息的内容，通信完成后调用closeSocket关闭这个对应accept到的socket，如果不再需要等待任何客户端连接，那么用

closeSocket关闭掉自身的socket。

客户端：建立socket，通过端口号和地址确定目标服务器，使用Connect连接到服务器，send发送消息，等待处理，通信完成后调用closeSocket关闭socket。

三、面向无连接的客户端/服务端模式：

与面向连接的网络连接相比，无连接的网络通信不需要在服务器与客户端之间建立连接。面向非连接的Socket通信是基于UDP的，服务器端不需要调用listen()和accept()函数来等待客户端的连接；客户端直接向服务器发送数据。

实验演示

c++实现客户服务器TCP通信示例代码：

服务端：**server.cpp**

```c
#include <stdio.h>
#include <winsock2.h>

#pragma comment(lib,"ws2_32.lib")

int main(int argc, char* argv[])
{
    //初始化WSA
    WORD sockVersion = MAKEWORD(2,2);
    WSADATA wsaData;
    if(WSAStartup(sockVersion, &wsaData)!=0)
    {
        return 0;
    }

    //创建套接字
    SOCKET slisten = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if(slisten == INVALID_SOCKET)
    {
        printf("socket error !");
        return 0;
    }

    //绑定IP和端口
    sockaddr_in sin;
    sin.sin_family = AF_INET;
    sin.sin_port = htons(8888);
    sin.sin_addr.S_un.S_addr = INADDR_ANY;
    if(bind(slisten, (LPSOCKADDR)&sin, sizeof(sin)) == SOCKET_ERROR)
    {
        printf("bind error !");
    }

    //开始监听
    if(listen(slisten, 5) == SOCKET_ERROR)
    {
        printf("listen error !");
        return 0;
    }
```

```
    //循环接收数据
    SOCKET sClient;
    sockaddr_in remoteAddr;
    int nAddrlen = sizeof(remoteAddr);
    char revData[255];
    while (true)
    {
        printf("等待连接...\n");
        sClient = accept(slisten, (SOCKADDR *)&remoteAddr, &nAddrlen);
        if(sClient == INVALID_SOCKET)
        {
            printf("accept error !");
            continue;
        }
        printf("接受到一个连接：%s \r\n", inet_ntoa(remoteAddr.sin_addr));

        //接收数据
        int ret = recv(sClient, revData, 255, 0);
        if(ret > 0)
        {
            revData[ret] = 0x00;
            printf(revData);
        }

        //发送数据
        const char * sendData = "你好，TCP客户端！\n";
        send(sClient, sendData, strlen(sendData), 0);
        closesocket(sClient);
    }

    closesocket(slisten);
    WSACleanup();
    return 0;
}
```

客户端代码：client.cpp

```cpp
#include<WINSOCK2.H>
#include<STDIO.H>
#include<iostream>
#include<cstring>
using namespace std;
#pragma comment(lib, "ws2_32.lib")

int main()
{
	WORD sockVersion = MAKEWORD(2, 2);
	WSADATA data;
	if(WSAStartup(sockVersion, &data)!=0)
	{
		return 0;
	}
	while(true){
		SOCKET sclient = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
		if(sclient == INVALID_SOCKET)
		{
			printf("invalid socket!");
			return 0;
		}

		sockaddr_in serAddr;
		serAddr.sin_family = AF_INET;
		serAddr.sin_port = htons(8888);
		serAddr.sin_addr.S_un.S_addr = inet_addr("127.0.0.1");
		if(connect(sclient, (sockaddr *)&serAddr, sizeof(serAddr)) == SOCKET_ERROR)
		{  //连接失败
			printf("connect error !");
			closesocket(sclient);
			return 0;
		}

		string data;
		cin>>data;
		const char * sendData;
		sendData = data.c_str();   //string转const char*
		//char * sendData = "你好，TCP服务端，我是客户端\n";
```

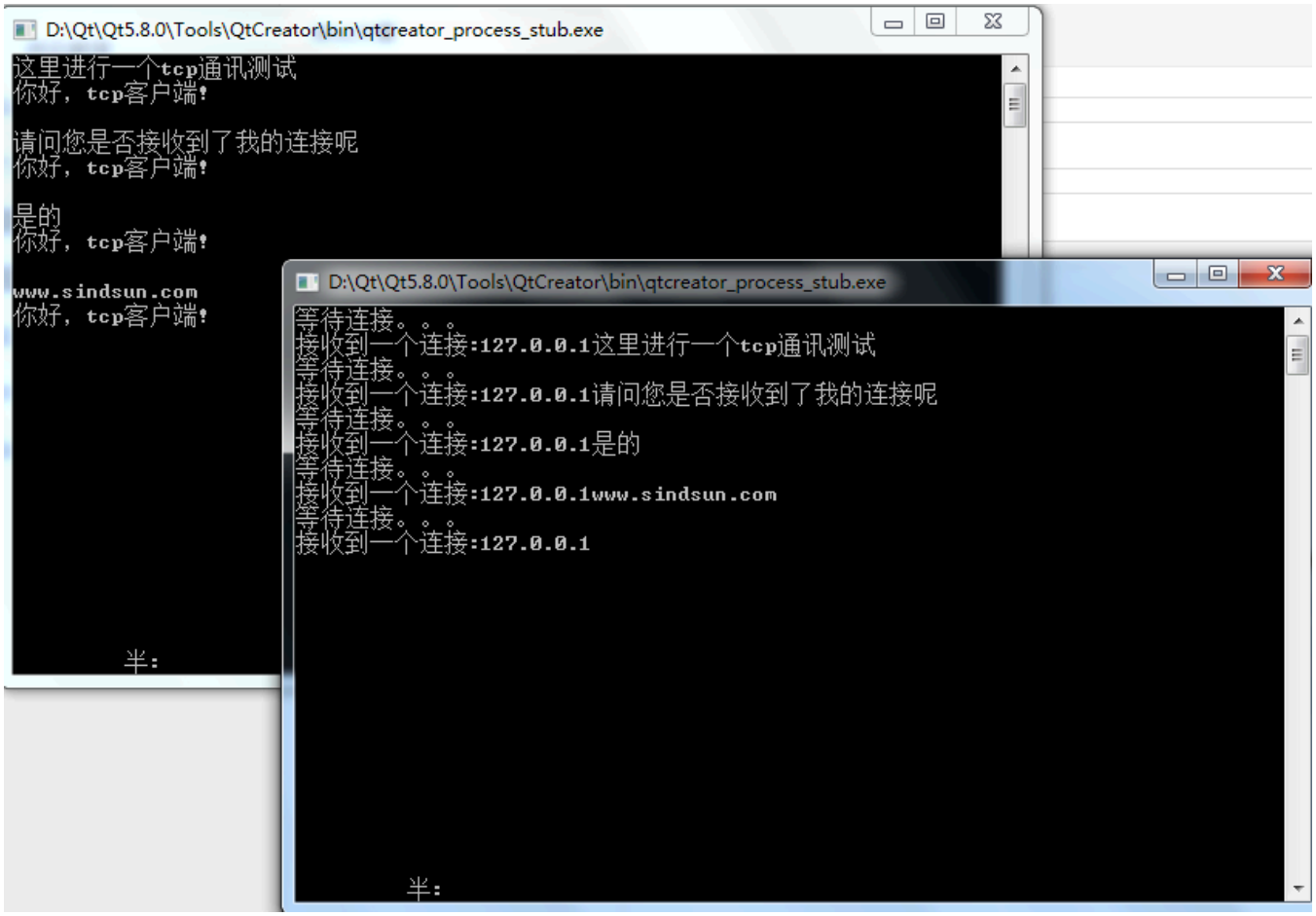
```
                send(sclient, sendData, strlen(sendData), 0);
                //send()用来将数据由指定的socket传给对方主机
                //int send(int s, const void * msg, int len, unsigned int flags)
                //s为已建立好连接的socket，msg指向数据内容，len则为数据长度，参数flags一般设0
                //成功则返回实际传送出去的字符数，失败返回-1，错误原因存于error

                char recData[255];
                int ret = recv(sclient, recData, 255, 0);
                if(ret>0){
                        recData[ret] = 0x00;
                        printf(recData);
                }
                closesocket(sclient);
        }


        WSACleanup();
        return 0;

}
```

运行结果：



其他介绍——Python socket编程

Python 编写 Server 端的步骤

第一步：创建 socket 对象

调用 socket 构造函数，如：

```
socket = socket.socket( family, type )
```

- family 参数代表地址家族，可以为 `AF_INET` 或者 `AF_UNIX`，AF_INET 家族包括 Internet 地址，AF_UNIX 家族则用于同一台机器上的进程间通信
- type 参数代表套接字类型，可以为 `SOCK_STREAM`（流套接字）或者 `SOCK_DGRAM`（数据报套接字）。

第二步：将 socket 绑定到指定地址

这是通过 socket 对象的 bind 方法来实现的：

```
socket.bind( address )
```

由 AF_INET 创建的套接字， address 必须是一个双元素无组，格式是 `( host, port )`，host 代表主机， port 代表端口号，如果端与正在使用，主机名不正确或者商品端口已被保留， bind 方法将引发 socket.error 异常。

第三步：使用 listen 套接字的 listen 方法接收连接请求

```
socket.listen( backlog )
```

backlog 指定最多允许多少个客户连接到服务器，它的值至少为 1，收到连接请求后，这些请求需要排队，如果队列已满，就拒绝请求。

第四步：服务器套接字通过 socket 的 accept() 方法等待客户请求一个连接

```
connection, address = socket.accept()
```

调用 accept 方法时， socket 会进入 “waiting” 状态，客户请求连接时，方法建立连接并返回服务器， accept 方法返回一个含有两个元素的元组（connection, address），第一个元素 connection 是新的 socket 对象，服务器必须通过它与客户通信；第二个元素 address 是客户的 Internet 地址。

第五步：处理

服务器和客户端通过 send 和 recv 方法通信（传输数据）。服务器调用 send ， 并采用字符串形式向客户发送信息，send 方法返回已发送的字符个数，服务器使用 recv 方法从客户端接收信息，调用 recv时，服务器必须指定一个整数，它对应于可通过本次方法调用来接收的最大数据量。 recv 方法在会进入“blocked”状态，最后返回一个字符串，用它表示接收到的数据，如果发送的数据量超过了 recv 所允许的数据会被截取，多余的数据将缓冲于接收端，以后调用 recv 时， 多余的数据会从缓冲区删除（以及上次调用 recv 以来，客户可能发送的其它任何数据）。

第六步：传输结束

服务器调用 socket 的 close 方法关闭连接

Python 编写客户端的步骤

第一步：创建一个 socket 以连接服务器

```
socket = socket.socket( family, type)
```

第二步：使用 socket 的 connect 方法连接服务器

对于 AF_INET 家族，连接格式如下：

```
socket.connect(( host, port ))
```

host 代表服务器的主机名或者 IP， port 代表服务器进程所绑定的端口号，如果连接成功，客户端就可以与服务器进行通信了，如果连接失败，会引发 socket.error异常

第三步：处理阶段

客户和服务器将通过 send 方法和 recv 方法通信

第四步：传输结果

客房端通过 socket 的 close 方法关闭连接

整个示例的完整代码

server.py 文件

```
#!/usr/bin/env python
# encoding: utf-8
"""
File: server.py
Created by Pan Tao on 2011-09-20.
Copyright (c) 2011 CosTony.Com. All rights reserved.
"""
def main():
import socket
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind(('localhost', 8000))
sock.listen(5)
while True:
connection, address = sock.accept()
try:
connection.settimeout(5)
buf = connection.recv(1024)
if buf == '1':
print 'Client {} connected to server!'.format(address)
connection.send('Welcome to server!')
else:
connection.send('Please go out!')
except socket.timeout:
print 'Time out!'
connection.close()
if __name__ == '__main__':
main()
```

client.py 文件

```
#!/usr/bin/env python
# encoding: utf-8
"""
File: client.py
Created by Pan Tao on 2011-09-20.
Copyright (c) 2011 CosTony.Com. All rights reserved.
"""
def main():
import socket
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(('localhost', 8000))
import time
time.sleep(2)
sock.send('1')
print sock.recv(1024)
sock.close()
if __name__ == '__main__':
main()
```

测试运行

我们先运行 server.py 以开启服务器

```
python server.py
```

然后运行 client.py 连接服务器

```
python client.py
```

运行结果：

Server 端将打印出：

```
Client ('127.0.0.1', 56933) connected to server!
```

客户端将打印出：

```
Welcome to server!
```

友情提醒

1. 不要直接使用文档提供的代码哦！
2. 只是通过终端实现socket编程推荐使用python，简单易上手。但是想使用可视化方式演示时不太推荐使用python，因为PyQt谁用谁知道（或者你有更好的解决办法）。